الجمهورية الجزائرية الديمقراطية الشعبية

وزارة التعليم العالي والبحث العلمي

République Algérienne Démocratique et Populaire

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université Benyoucef BENKHEDDA-Alger1

Faculté des Sciences

Département d'informatique

Projet de Fin d'Etude spour l'obtention du diplôme de la Licence en Informatique

Option : Systèmes Informatiques

Ingénierie des Systèmes d'Information et Logiciels

Thème

Development of a web application for IT infrastructure Documentation and Incident Knowledge Management

Encadré par :

- Mme **AGGOUNE Soumia**
- Mr **BECHIRI Nouredine**

Réalisé par :

- **FERSOUS Tarek**
- **FADLI El Amine**

Dedication

Abstract

MapIT is a web-based platform designed to centralize IT infrastructure documentation and incident knowledge management in a single, secure workspace. The project addresses the common operational problem of fragmented notes, duplicated troubleshooting, and weak traceability between incidents and the knowledge used to resolve them. The proposed solution combines a structured data model, role-based access control, and a clear user interface to support incident registration, document maintenance, and knowledge reuse. The system is implemented as a full-stack application with a separated frontend, backend API, and relational database, making it suitable for teams that need a lightweight but disciplined documentation environment.

Keywords: IT infrastructure, incident management, knowledge management, documentation platform, role-based access control.

Acknowledgements

We would like to express our sincere gratitude to our supervisors, instructors, and all members of the department who supported the preparation of this project. Their guidance, technical feedback, and academic rigor helped us structure the work, refine the design choices, and maintain a clear line between theoretical analysis and practical realization. We also thank everyone who contributed directly or indirectly to the development of MapIT through advice, review, and encouragement.

Contents

Dedication i

Abstract ii

Acknowledgements iii

List of Figures vii

List of Tables viii

List of Abbreviations ix

I Theoretical Part 1

1 General Context and Introduction 2
1.1 Introduction . . . 2
1.2 Host Organization: SNTF . . . 2
1.2.1 Short presentation . . . 2
1.2.2 Visual identity . . . 3
1.2.3 DSI organigramme . . . 3
1.2.4 Why SNTF is relevant to MapIT . . . 5
1.3 General Context . . . 5
1.4 Problem Statement . . . 5
1.4.1 The operational gap . . . 5
1.4.2 Why the gap matters . . . 6
1.4.3 What is specific to MapIT . . . 6
1.5 Motivations and Project Relevance . . . 6
1.5.1 Academic motivation . . . 6
1.5.2 Professional motivation . . . 7
1.6 Our Role as Computer Science Students Specializing in Computer Systems . . . 7
1.6.1 Academic role . . . 7
1.6.2 Technical role . . . 7
1.7 Project Objectives and Scope . . . 8
1.7.1 Functional objectives . . . 8
1.7.2 Scope of the first phase . . . 8
1.7.3 Planning and work organization . . . 8
1.8 Chapter Conclusion . . . 8

2 Study of Existing Solutions 9
2.1 Introduction . . . 9
2.2 Methodology for Comparing Existing Solutions . . . 9
2.3 Review of Existing Platforms . . . 10
2.4 Comparative Analysis . . . 10
2.5 Limitations of the Existing Approaches . . . 11
2.6 Positioning of MapIT . . . 12
2.7 Chapter Conclusion . . . 12

3 IT Infrastructure Documentation and Incident Knowledge Management **13**
3.1 Introduction . . . 13
3.2 Information Systems as a Knowledge Infrastructure . . . 13
3.2.1 Definition of an information system . . . 13
3.2.2 Definition of IT documentation . . . 14
3.2.3 From data to operational knowledge . . . 14
3.2.4 Why the concept matters for MapIT . . . 15
3.3 Knowledge Management in IT Operations . . . 15
3.3.1 Knowledge capture and reuse . . . 15
3.3.2 Knowledge quality and traceability . . . 15
3.3.3 Why this is important for IT teams . . . 15
3.4 Incident Tracking and Relationship Mapping . . . 16
3.4.1 Incident tracking . . . 16
3.4.2 Relationship mapping . . . 16
3.4.3 Why both concepts are essential . . . 16
3.5 Applying the Theory to MapIT . . . 17
3.5.1 The MapIT domain model . . . 17
3.5.2 How the theory appears in the interface . . . 17
3.5.3 Consistency of the theoretical path . . . 17
3.6 Chapter Conclusion . . . 17

II Practical Part **19**

4 Conception **20**
4.1 Introduction . . . 20
4.2 Requirement Elicitation . . . 20
4.3 Functional Requirements . . . 20
4.4 Non-Functional Requirements . . . 21
4.5 High-Level Architecture . . . 21
4.6 Module Decomposition and User Journeys . . . 22
4.7 Design Decisions and Constraints . . . 22
4.8 Chapter Conclusion . . . 23

5 Modeling **24**
5.1 Introduction . . . 24
5.2 Actors and Use Cases . . . 24
5.3 Use Case Specifications . . . 26
5.4 Sequence Diagrams . . . 26
5.5 Class Diagram and Domain Model . . . 29
5.6 Data Model and Relational Schema . . . 31
5.7 Optional Complementary Diagrams . . . 31
5.8 Chapter Conclusion . . . 32

6 Implementation **33**
6.1 Introduction . . . 33
6.2 Technical Stack and System Architecture . . . 33
6.2.1 Monorepo structure . . . 33
6.2.2 Web implementation stack . . . 34
6.2.3 Backend implementation stack . . . 34
6.2.4 Data and persistence layer . . . 34
6.3 Authentication and Access Control . . . 35
6.3.1 Login flow . . . 35
6.3.2 Session and proxy handling . . . 35
6.3.3 Role-based and permission-based access control . . . 36
6.4 Backend Modules and Business Functions . . . 36
6.4.1 Incidents . . . 36
6.4.2 Assets and relationships . . . 37
6.4.3 Problems and knowledge reuse . . . 37

6.4.4 Search, reports, settings, and audit logs . . . 37
6.5 Realized User Interfaces . . . 38
6.5.1 Login and shared layout . . . 38
6.5.2 Dashboard . . . 38
6.5.3 Incidents . . . 39
6.5.4 Assets . . . 40
6.5.5 Problems, relationships, search, reports, and settings . . . 40
6.6 Deployment and Execution Notes . . . 42
6.6.1 Local execution . . . 42
6.6.2 Environment and configuration . . . 43
6.6.3 Production orientation . . . 43
6.7 Validation and Testing . . . 43
6.7.1 Backend checks . . . 43
6.7.2 Frontend checks . . . 43
6.8 Chapter Conclusion . . . 43

General Conclusion **45**

List of Figures

1.1 Official logo of SNTF . . . 3
1.2 Organigramme of the SNTF DSI . . . 4

4.1 High-level architecture of MapIT . . . 22

5.1 Use case diagram of MapIT . . . 25
5.2 Sequence diagram: authentication flow . . . 27
5.3 Sequence diagram: incident management flow . . . 28
5.4 Sequence diagram: global search flow . . . 29
5.5 Class diagram of the MapIT domain model . . . 30
5.6 Activity diagram: RBAC decision flow . . . 32

6.1 Login page . . . 35
6.2 Access control management interface . . . 36
6.3 Audit logs interface . . . 38
6.4 Dashboard interface . . . 39
6.5 Incidents management interface . . . 39
6.6 Assets inventory interface . . . 40
6.7 Problem and knowledge base interface . . . 41
6.8 Relationship map interface . . . 41
6.9 Global search interface . . . 42
6.10 Reports interface . . . 42

List of Tables

2.1 Feature comparison of existing solutions 11

3.1 Impact of documentation on IT operations 14

List of Abbreviations

- ITSM – Information Technology Service Management
- CMDB – Configuration Management Database
- UML – Unified Modeling Language
- API – Application Programming Interface
- RBAC – Role-Based Access Control
- JWT – JSON Web Token

Part I

Theoretical Part

Chapter 1

General Context and Introduction

1.1 Introduction

MapIT is a full-stack platform designed to support IT infrastructure documentation, incident handling, asset inventory, relationship mapping, and knowledge reuse. Unlike a generic document repository, the project is built around the operational lifecycle of an IT support team: identify an asset, relate it to other components, track incidents against it, preserve the resulting knowledge, and make that knowledge searchable for later reuse. This chapter introduces the project context, states the problem precisely, and explains why the work is academically and professionally relevant.

1.2 Host Organization: SNTF

1.2.1 Short presentation

The host organization for this project is the *Société nationale des transports ferroviaires* (SNTF), the Algerian public railway company responsible for passenger and freight transport, as well as the operation and maintenance of the national rail network. According to the company presentation and public references, SNTF was created in 1976 and later became an EPIC in 1990. Its headquarters are in Algiers, and its activity covers rail operations across the country [5, 6].

1.2.2 Visual identity

Figure 1.1 presents the official SNTF logo used as a visual reference for the host organization.

Figure 1.1: Official logo of SNTF

1.2.3 DSI organigramme

The DSI organigramme shows how the Information Systems Department is organized around several complementary missions: project management, network administration, systems development, web activities, database administration, security-oriented messaging and workstation support, and computer maintenance. This structure is useful for understanding where an internal documentation platform such as MapIT can support coordination between technical teams.

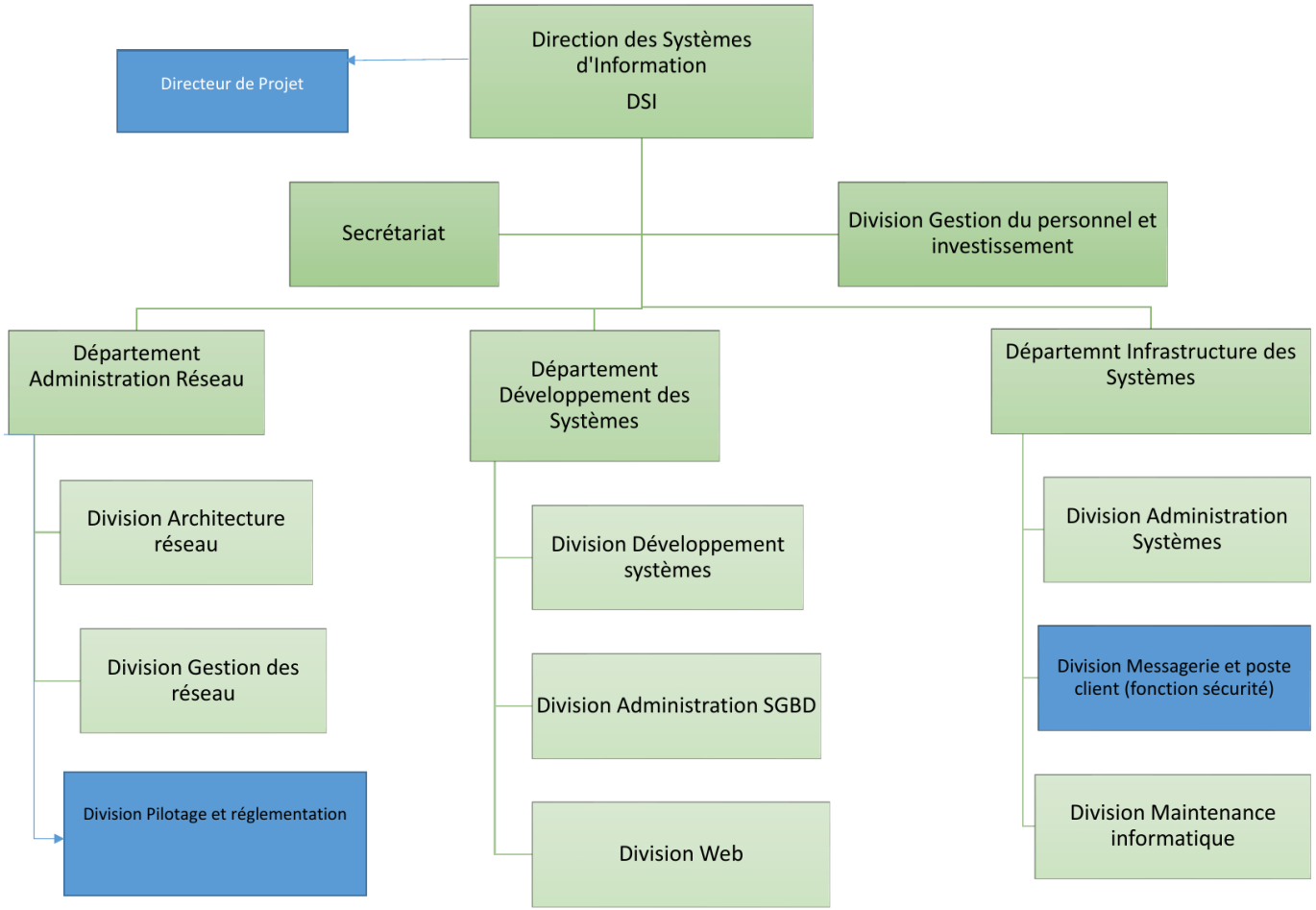

Figure 1.2: Organigramme of the SNTF DSI

1.2.4 Why SNTF is relevant to MapIT

SNTF is a natural context for a platform like MapIT because railway operations depend on infrastructure documentation, service continuity, traceability, and coordination between technical teams. The organization manages a geographically distributed transport network, which makes structured knowledge and clear visibility over assets, incidents, and dependencies especially valuable. MapIT is therefore aligned with the documentation needs of an operational and asset-intensive enterprise.

1.3 General Context

Modern IT infrastructures are distributed across servers, virtual machines, services, network devices, knowledge articles, and incident records. In MapIT, this complexity is visible in the codebase itself: the web app exposes dedicated routes for `/assets`, `/incidents`, `/problems`, `/relationships`, `/search`, `/reports`, and `/settings`, while the backend defines matching API modules for each domain. A platform in this context must do more than display data; it must organize technical memory so that support teams can understand dependencies, resolve incidents faster, and preserve the reasoning behind corrective actions.

1.4 Problem Statement

1.4.1 The operational gap

The main problem solved by MapIT is the absence of a centralized and visual knowledge management environment for IT assets and incidents. In many real teams, asset data is kept in one tool, incident records in another, and solutions in a separate document or chat history. That fragmentation prevents a support engineer from seeing the full operational picture: which component failed, what depends on it, which incident already documented the failure, and which solution should be reused.

1.4.2 Why the gap matters

When documentation is fragmented, the same diagnostic effort is repeated several times, response time increases, and the organization loses institutional memory. It also becomes hard to audit decisions, assign responsibility, or track the evolution of a problem from its first symptom to its final resolution. In MapIT, this issue is addressed by connecting incidents to assets, problems, documents, and relationships, while also logging access and permission-sensitive actions in an audit trail.

1.4.3 What is specific to MapIT

MapIT does not aim to be a generic content management platform. It addresses an IT operations problem: the need to centralize technical knowledge in a form that is both textual and visual. The relationship map, the global search page, the incident knowledge suggestions, the asset/problem linkage, and the role-based restrictions on editing are all designed to solve this exact problem. The project therefore focuses on the lack of centralized, visual knowledge management for IT assets, which is the real operational gap behind the application.

1.5 Motivations and Project Relevance

1.5.1 Academic motivation

The project is academically valuable because it combines the main themes expected in a Computer Systems specialization: information systems analysis, relational data modeling, API design, authentication, permissions, and a modern web user interface. It follows a clear progression from problem definition to modeling and implementation, which is appropriate for a final-year project of this scope.

1.5.2 Professional motivation

Professionally, MapIT reflects the kind of system used by technical support teams that need to keep track of infrastructure knowledge, incident history, and operational dependencies. The implementation includes a dashboard with summary metrics, an asset inventory, an incident lifecycle workflow, a problem/solution knowledge layer, a graph-based relationship view, reporting, and settings with access control and audit logs. This is the behavior expected from an operational knowledge platform, not from a simple brochure website.

1.6 Our Role as Computer Science Students Specializing in Computer Systems

1.6.1 Academic role

As Computer Science students specializing in Computer Systems, our role was to turn a real operational need into a formal software architecture and then into a working implementation. This involved understanding the domain, defining the data model, specifying the access rules, and implementing the frontend and backend in a way that remains coherent and maintainable.

1.6.2 Technical role

The code shows that our technical role was broader than writing pages: we had to design the Prisma schema, implement JWT-based authentication with LDAP support in production and dev credentials in development, enforce permissions through middleware, and make the frontend consume the API through a Next.js proxy route. These tasks reflect a systems-oriented skill set because they connect infrastructure understanding with application engineering.

1.7 Project Objectives and Scope

1.7.1 Functional objectives

The immediate objectives of MapIT are to authenticate users, show an operational dashboard, manage assets, track incidents, document reusable problems and solutions, display infrastructure relationships, search across all knowledge types, and provide reporting and settings screens. The codebase confirms that each of these capabilities has a dedicated route and user interface.

1.7.2 Scope of the first phase

The current phase focuses on the essential operational workflow. It already includes login, dashboard summaries, incident list and detail views, asset inventory, knowledge/problem pages, relationship visualization, global search, reports, and settings. More advanced features such as richer analytics, automated workflows, or deeper notification logic can be introduced later without changing the core model.

1.7.3 Planning and work organization

A Gantt chart can be inserted here to show the sequence of work: analysis, study of existing solutions, theoretical study, conception, modeling, implementation, and validation. This helps the jury understand the development chronology of the project.

1.8 Chapter Conclusion

This chapter established that MapIT solves a concrete infrastructure knowledge problem: the absence of a centralized, visual, and role-aware place to manage assets, incidents, and reusable technical knowledge. The next chapter reviews the theoretical concepts that justify this architecture, especially information systems, knowledge management, incident tracking, and relationship mapping.

Chapter 2

Study of Existing Solutions

2.1 Introduction

This chapter reviews existing tools and platforms that address part or all of the problem space targeted by MapIT. The purpose of the study is not only to list popular software, but also to understand how current products handle documentation, incident traceability, collaboration, and knowledge reuse. The conclusions drawn here serve as a foundation for the design choices presented in the following chapters.

2.2 Methodology for Comparing Existing Solutions

The comparison is based on criteria that are directly relevant to an IT documentation and knowledge management platform. These criteria include the ability to organize knowledge in a structured way, the quality of search and retrieval, support for incident traceability, collaboration features, access control, auditability, deployment flexibility, and licensing constraints. Using these criteria makes it possible to compare products with different primary goals on a common functional basis.

2.3 Review of Existing Platforms

Platforms such as ServiceNow, Jira Service Management, Confluence, GLPI, BookStack, Notion, SharePoint, and IT Glue show that the market already offers mature solutions for ticketing, documentation, or knowledge sharing. However, these platforms usually optimize one dominant workflow: service desk management, collaborative writing, enterprise content management, or generic note organization. For a project like MapIT, the key question is therefore not whether such tools are powerful, but whether they provide a focused and lightweight environment where infrastructure documentation and incident knowledge can be managed together without unnecessary complexity.

2.4 Comparative Analysis

The comparison table inserted in this section should make the differences visible at a glance. In many cases, enterprise tools are stronger in governance and workflow control, while lightweight tools are easier to adopt but weaker in traceability and structured access control. The analysis should therefore emphasize that the value of MapIT lies in balancing clarity, focused functionality, and academic feasibility rather than trying to reproduce the full breadth of a large commercial platform.

Solution	**Main strengths**	**Main limits**	**Relevance for MapIT**	
ServiceNow	Mature ITSM workflows, governance, and automation.	Heavy, costly, and broader than the project scope.	Benchmark for rigor and access control.	
GLPI	Open-source service desk with asset management.	Less oriented to relationship maps and reusable knowledge.	Useful for compact IT support comparison.	
Confluence / BookStack	Strong structured documentation and collaboration.	Keeps documentation apart from incidents and dependencies.	Good reference for knowledge-base organization.	
Notion	Flexible and easy for note-taking.	Weak structure, traceability, and IT-specific workflow control.	Shows limits of generic knowledge tools.	
MapIT	Integrates assets, incidents, problems, relationships, search, and access control.	Deliberately narrower than enterprise suites.	Target solution of the study.	

Table 2.1: Feature comparison of existing solutions

2.5 Limitations of the Existing Approaches

The reviewed approaches reveal several recurring limitations. Some tools are expensive or too heavy for a compact academic deployment, while others are flexible but too generic to enforce a meaningful relationship between incidents and technical knowledge. In addition, documentation often remains separated from operational history, which means that the learning value of previous incidents is not fully preserved in everyday practice.

2.6 Positioning of MapIT

MapIT is positioned as a focused web application that covers the essential needs of infrastructure documentation and incident knowledge management without the overhead of a large enterprise suite. Its contribution is not to replace the market leaders, but to demonstrate a coherent solution tailored to the specific workflow of an IT team that needs controlled access, a clear relational model, and a direct connection between incidents and reusable knowledge.

2.7 Chapter Conclusion

This chapter showed that existing solutions address the problem only partially or at a broader scale than what the project requires. The comparison supports the decision to build MapIT as a compact, role-aware, and documentation-centric platform. The next chapter introduces the theoretical concepts that define the project domain.

Chapter 3

IT Infrastructure Documentation and Incident Knowledge Management

3.1 Introduction

This chapter is the theoretical bridge between the study of existing solutions and the practical implementation of MapIT. The purpose is to define, in formal academic English, the concepts that make the project meaningful: information systems, knowledge management, incident tracking, and relationship mapping. These concepts are not theoretical ornaments; they correspond directly to the way MapIT is modeled in the codebase.

3.2 Information Systems as a Knowledge Infrastructure

3.2.1 Definition of an information system

An information system is an organized combination of people, processes, data, and technology that transforms raw data into usable information and supports operational decision-making [4]. In MapIT, this definition is visible in the separation between the

React/Next.js user interface, the Express backend API, the PostgreSQL database, and the permission middleware. The system is therefore not just a website; it is a knowledge infrastructure for IT operations.

3.2.2 Definition of IT documentation

IT documentation can be defined as the structured and continuously maintained set of records that describes infrastructure components, operating procedures, dependencies, incidents, and decisions. Its role is to preserve operational memory so that technical teams can understand what exists, how it behaves, and what was done previously to maintain or restore it. In an IT service context, documentation is valuable only when it is accurate, searchable, and linked to the real workflow of the organization [1, 3].

Table 3.1 summarizes the operational effect of documentation quality on daily IT work.

Feature	**With Documentation**	**Without Documentation**
Troubleshooting	Fast, efficient	Slow, error-prone
Onboarding	Clear, structured	Confusing, inconsistent
Compliance	Easier to prove	Risk of violations
Knowledge Sharing	Smooth	Dependent on individuals
System Reliability	Higher	Lower

Table 3.1: Impact of documentation on IT operations

3.2.3 From data to operational knowledge

MapIT stores data such as asset names, incident titles, problem descriptions, relationship types, and audit entries. This data becomes information when it is structured into pages, filters, status flows, or summary reports. It becomes operational knowledge when a technician can reuse it to answer a new incident, understand a dependency, or validate a corrective action. This progression is exactly why the application includes incidents, problems, documents, search, and feedback on knowledge suggestions.

3.2.4 Why the concept matters for MapIT

The application is designed to preserve the technical memory of the infrastructure. A dashboard, a relationship graph, a search index, and a report summary are all mechanisms that help the user see the system as a whole. In theoretical terms, MapIT is an information system that supports knowledge capture and knowledge retrieval in the same operational flow.

3.3 Knowledge Management in IT Operations

3.3.1 Knowledge capture and reuse

Knowledge management is the discipline of capturing experience, organizing it, validating it, and making it reusable [3]. In MapIT, the reusable layer is represented by problems with solutions, incidents with suggested knowledge, and documents that can be consulted later. The `IncidentKnowledgeFeedback` entity makes this process explicit by allowing users to say whether a suggested item was helpful.

3.3.2 Knowledge quality and traceability

Operational knowledge must be reliable, not merely available. That is why MapIT keeps audit logs, permission checks, and user-specific settings. The platform must know who changed a record, when it was changed, and under which permission level the action was executed. Without that traceability, the knowledge base would lose credibility in a real operational context.

3.3.3 Why this is important for IT teams

In an IT support environment, a good answer is not enough unless it can be found quickly and trusted. MapIT addresses this by indexing assets, problems, incidents, and documents through global search, by separating read access from management actions, and by exposing the operational state through a dashboard and reports. This is knowledge

management as an operational practice, not just as a repository.

3.4 Incident Tracking and Relationship Mapping

3.4.1 Incident tracking

Incident tracking is the formal recording of an incident from its opening to its closure, aligned with service management principles [1, 2]. In the backend, MapIT uses a controlled incident lifecycle with the statuses `OPEN`, `IN_PROGRESS`, `BLOCKED`, `RESOLVED`, and `CLOSED`. The detail page also supports priority, assignment, team ownership, and changes to status. This is important because it reflects the lifecycle discipline expected in modern service operations.

3.4.2 Relationship mapping

Relationship mapping is the representation of dependency between infrastructure elements. In MapIT, relationships are stored as directed links between assets and visualized in a graph where the user can drag nodes, zoom, filter asset types, and inspect the connections. This feature is not decorative: it allows the user to reason about impact, identify upstream and downstream dependencies, and understand which asset relationships are affected by a failure.

3.4.3 Why both concepts are essential

Incident tracking gives the temporal view of a problem; relationship mapping gives the structural view of the infrastructure. Modern IT environments need both because a single incident can affect multiple assets, and a single asset can be implicated in multiple incidents over time. MapIT combines these two views so that the user can move from a symptom to a dependency to a reusable solution.

3.5 Applying the Theory to MapIT

3.5.1 The MapIT domain model

The Prisma schema shows how the theoretical concepts become concrete entities: `User`, `Incident`, `Document`, `Asset`, `Problem`, `Relationship`, `Role`, `Permission`, `AuditLog`, `UserSettings`, and `IncidentKnowledgeFeedback`. These objects reflect the true domain of the project and prove that MapIT is centered on operational knowledge rather than on general content management.

3.5.2 How the theory appears in the interface

The theory is visible in the frontend as well. The login page controls entry, the dashboard summarizes the environment, the incident detail page supports lifecycle actions, the asset pages show related problems and relationships, the search page unifies access to all knowledge, and the settings page exposes access control and audit logs. This is the practical embodiment of the theoretical concepts defined in the chapter.

3.5.3 Consistency of the theoretical path

The theoretical path of this chapter moves from the definition of the domain to analysis, modeling, and realization. MapIT follows this logic through a practical IT infrastructure knowledge-management use case. The academic rigor is preserved by defining the concepts, justifying the approach, and showing how the implementation proves the model.

3.6 Chapter Conclusion

MapIT is grounded in established notions of information systems and knowledge management, but it applies them to a very specific IT operations problem: keeping infrastructure knowledge centralized, visual, and actionable. Incident tracking and relationship mapping are therefore core academic concepts for the project, not side features. The next chapter will show how these concepts were translated into the concrete software architecture and

user-facing pages.

Part II

Practical Part

Chapter 4

Conception

4.1 Introduction

This chapter converts the theoretical understanding of the domain into a concrete design for MapIT. The goal is to define what the system must do, how it should be organized, and which constraints guide the technical choices. The conception phase therefore bridges the gap between the problem statement and the formal modeling of the solution.

4.2 Requirement Elicitation

The expected needs of the platform were identified by examining the everyday tasks performed in documentation and incident-oriented IT workflows. This includes the need to consult knowledge quickly, register incidents with enough detail to be useful later, and keep the interface simple enough for non-specialists to use. The requirements were then separated into functional needs, which describe what the system must do, and non-functional needs, which describe how well it must do it.

4.3 Functional Requirements

The main functional requirements of MapIT are straightforward: users must authenticate securely, consult a dashboard, register incidents, create and update documentation entries,

search content, and perform actions according to their role. Administrators must be able to manage sensitive operations, while standard users should still have access to the knowledge base and incident records required for their work. This definition keeps the first release focused on core operational value rather than peripheral features.

4.4 Non-Functional Requirements

Several non-functional qualities are important for this type of system. Security is necessary because the platform stores operational information and controls access to it; usability matters because the knowledge base must be easy to navigate under time pressure; maintainability is important because the application should remain understandable as it evolves; and traceability is needed so that users can trust the history of the data. Performance and reliability also matter, even in an academic prototype, because slow search or unstable access would immediately reduce adoption.

4.5 High-Level Architecture

MapIT follows a simple three-tier architecture composed of a frontend, a backend API, and a relational database. The frontend is responsible for the user experience, the backend handles business logic and authorization, and the database stores the structured records that support incidents, documents, and users. This separation improves maintainability and makes the system easier to evolve because each layer has a clear responsibility.

Figure 4.1 presents the concrete architecture used in the project, including the web client, API server, and persistence layer.

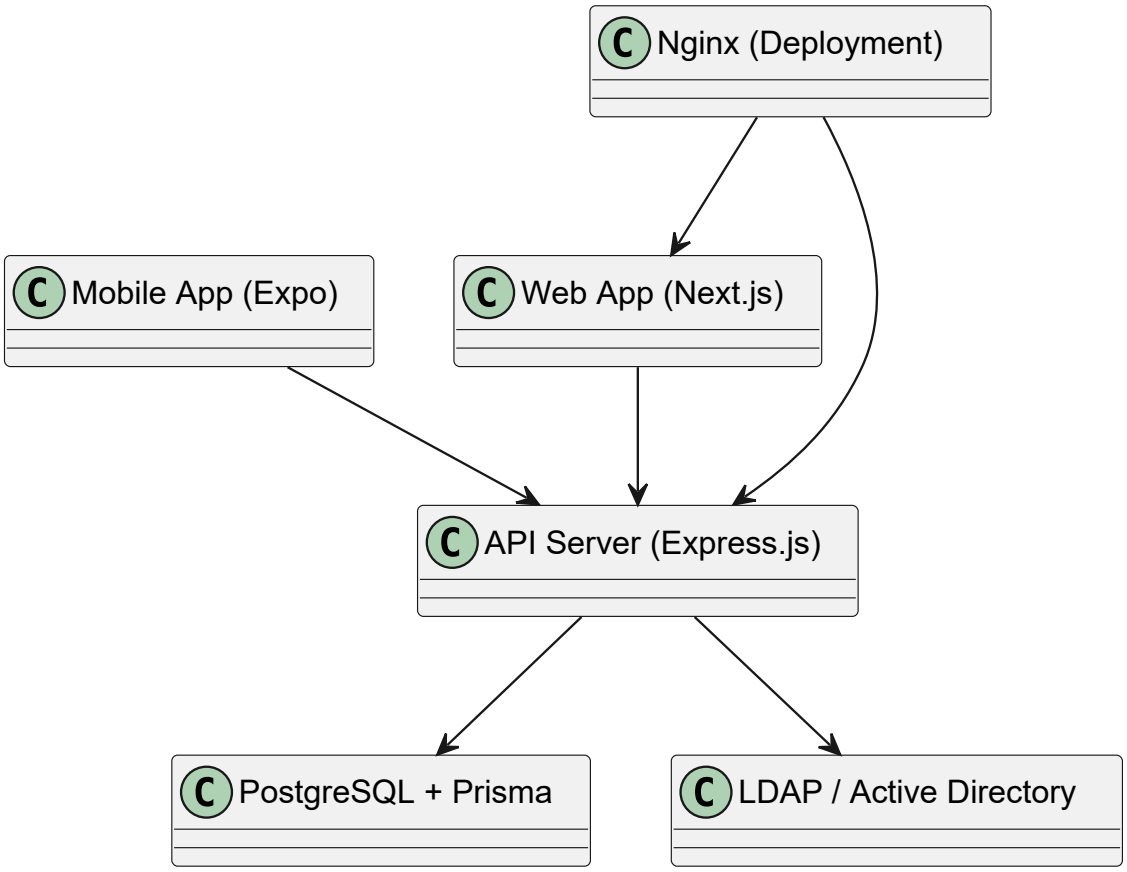

Figure 4.1: High-level architecture of MapIT

4.6 Module Decomposition and User Journeys

The system is divided into modules that reflect the main business activities of the platform: authentication, dashboard visualization, incident management, document management, and administration. A typical user journey begins at login, continues with a dashboard overview, and then leads to either the creation of a new record or the consultation of an existing one. This modular design keeps the application understandable and allows each user role to follow a path that matches its responsibilities.

4.7 Design Decisions and Constraints

The design choices are driven by a need for data consistency, controlled access, and easy navigation. A centralized data model avoids duplication between incidents and

documents, while role-based rules prevent unauthorized modification of critical records. At the same time, the interface must remain clean and direct so that the system can be used as a practical knowledge tool rather than as a purely theoretical database.

4.8 Chapter Conclusion

The conception phase establishes a clear functional direction for the application and frames the main constraints that the implementation must respect. It shows that MapIT is intentionally focused on the workflows that matter most for incident-oriented documentation. The next chapter formalizes this design through UML and data modeling artifacts.

Chapter 5

Modeling

5.1 Introduction

This chapter formalizes MapIT through analysis and modeling artifacts. The diagrams and schemas presented here translate the conception choices into a precise structural view of the system. They are important because they make the behavior of the application easier to verify before implementation begins.

5.2 Actors and Use Cases

The use case model should identify the principal actors of the system, such as a standard user and an administrator, and show how each role interacts with the application. The use case diagram provides a global view of the services offered by MapIT, including authentication, incident management, document management, and administrative operations. This representation helps the reader understand the scope of the system before entering into detailed scenarios.

Figure 5.1 summarizes the interactions between users and the core services of the platform.

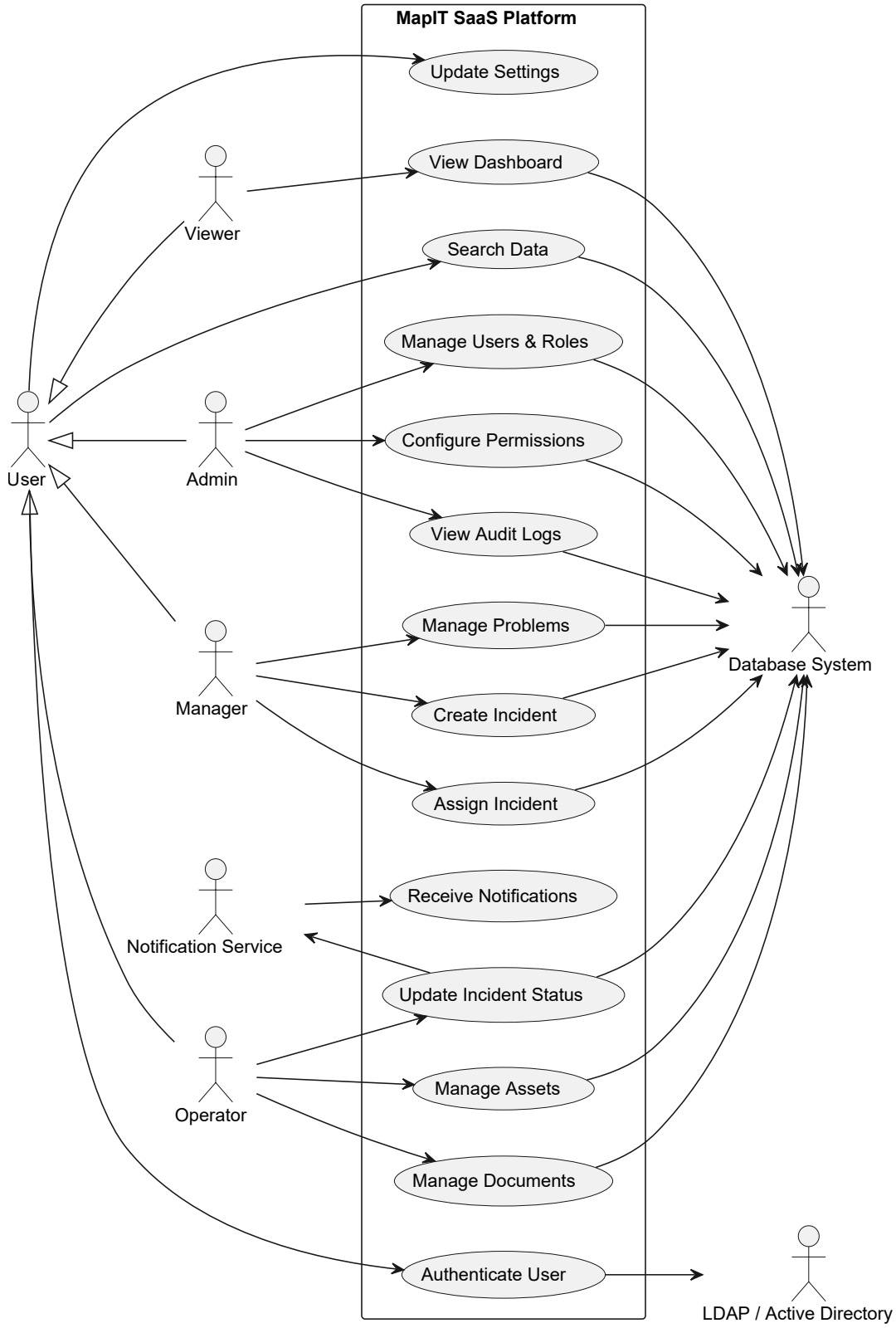

Figure 5.1: Use case diagram of MapIT

5.3 Use Case Specifications

Each major use case should then be described textually in terms of goal, actor, preconditions, main steps, alternate paths, and postconditions. This level of detail is important because it shows how the system should behave when a user logs in, creates a record, edits content, or performs a restricted action. The textual specification also serves as a useful reference during development and testing.

5.4 Sequence Diagrams

Sequence diagrams are used to describe the dynamic behavior of the system for representative scenarios such as authentication, incident creation, and document consultation. They show the message flow between the user interface, the API layer, the business logic, and the database, making it easier to verify that the architecture supports the intended user journeys. In an academic report, these diagrams also help demonstrate that the implementation follows a coherent interaction pattern.

The authentication interaction flow is shown in Figure 5.2.

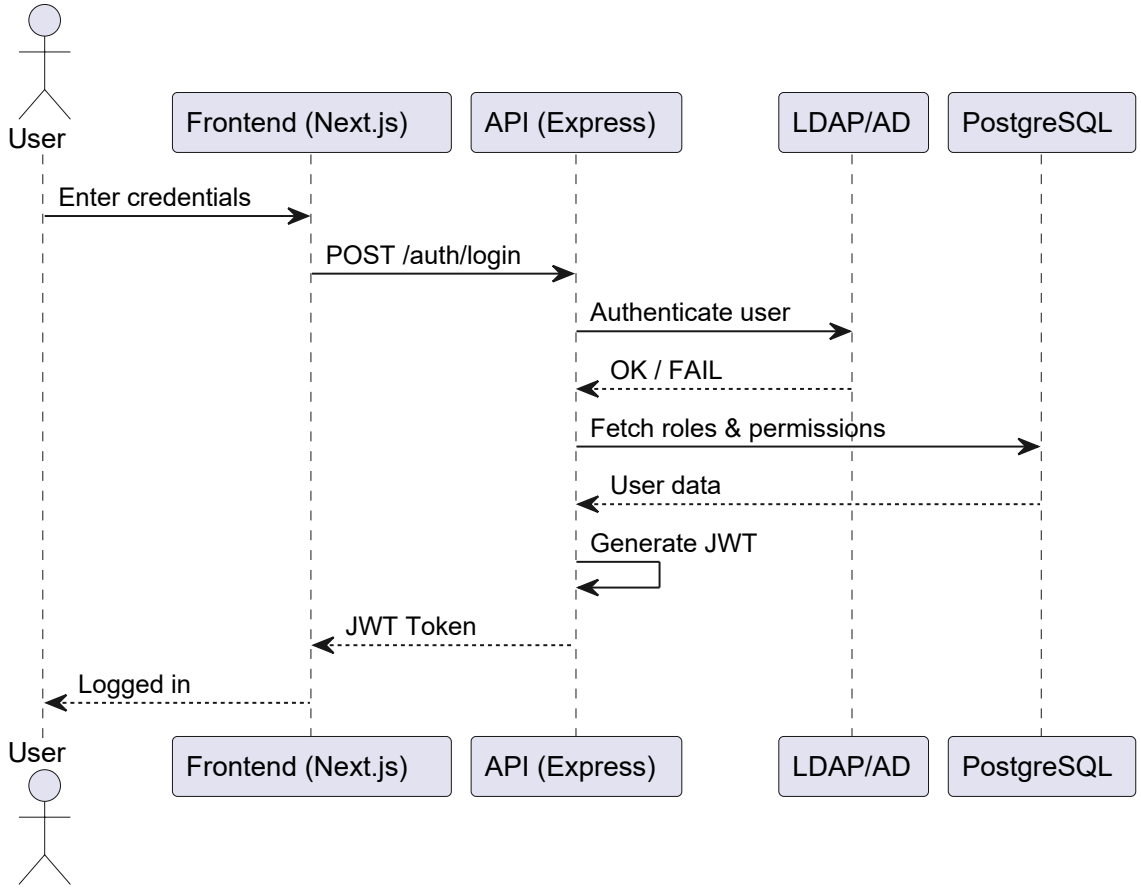

Figure 5.2: Sequence diagram: authentication flow

The lifecycle of incident processing is detailed in Figure 5.3.

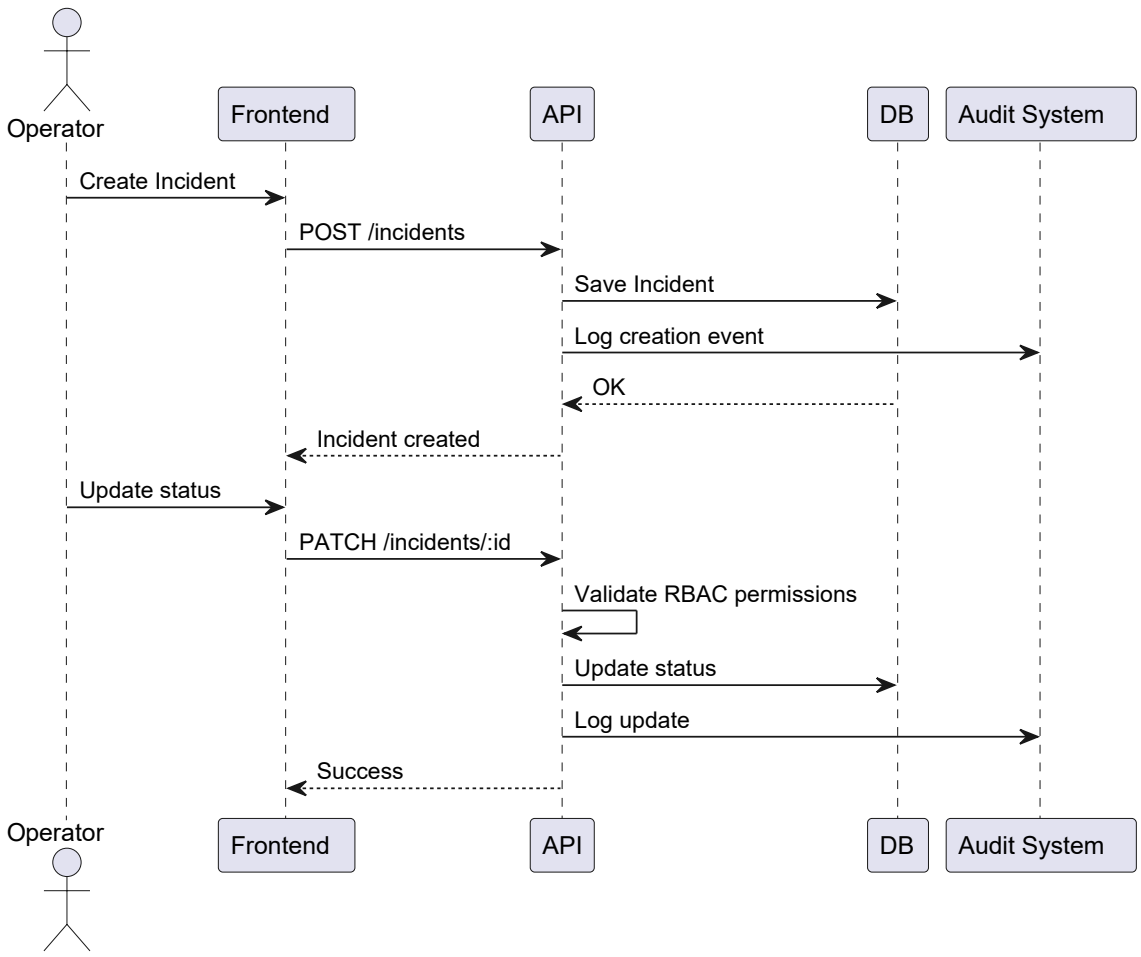

Figure 5.3: Sequence diagram: incident management flow

The global search interaction, which aggregates multiple data domains, is presented in Figure 5.4.

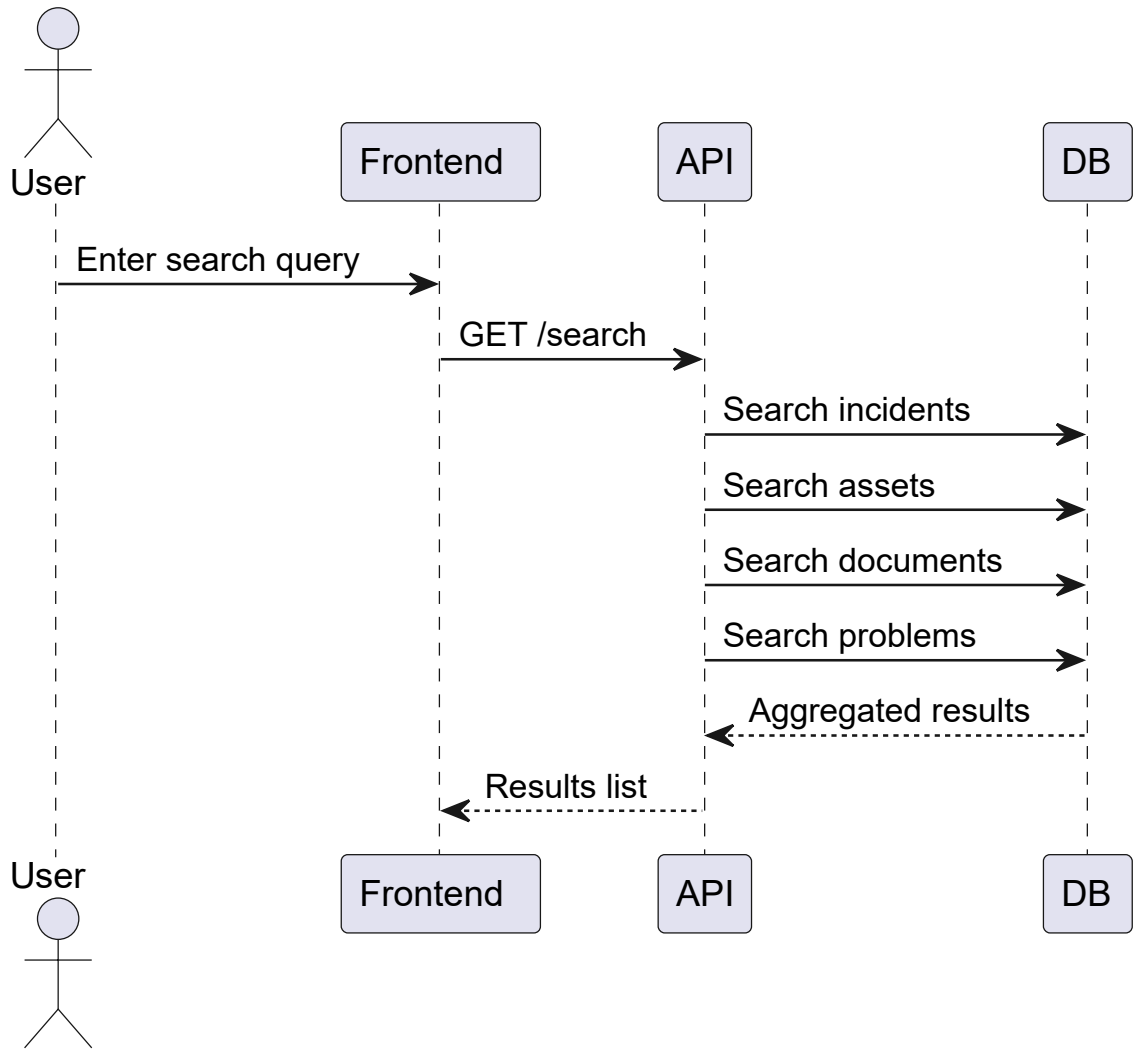

Figure 5.4: Sequence diagram: global search flow

5.5 Class Diagram and Domain Model

The class diagram captures the static domain structure of MapIT and presents the main entities of the application. Typical classes include User, Role, Incident, Document, Category, Comment, and AuditLog, together with the relationships that connect them. The diagram should make it clear which objects own data, which objects reference others, and how the domain supports both documentation and traceability.

Figure 5.5 illustrates the domain model used to implement these entities and their associations.

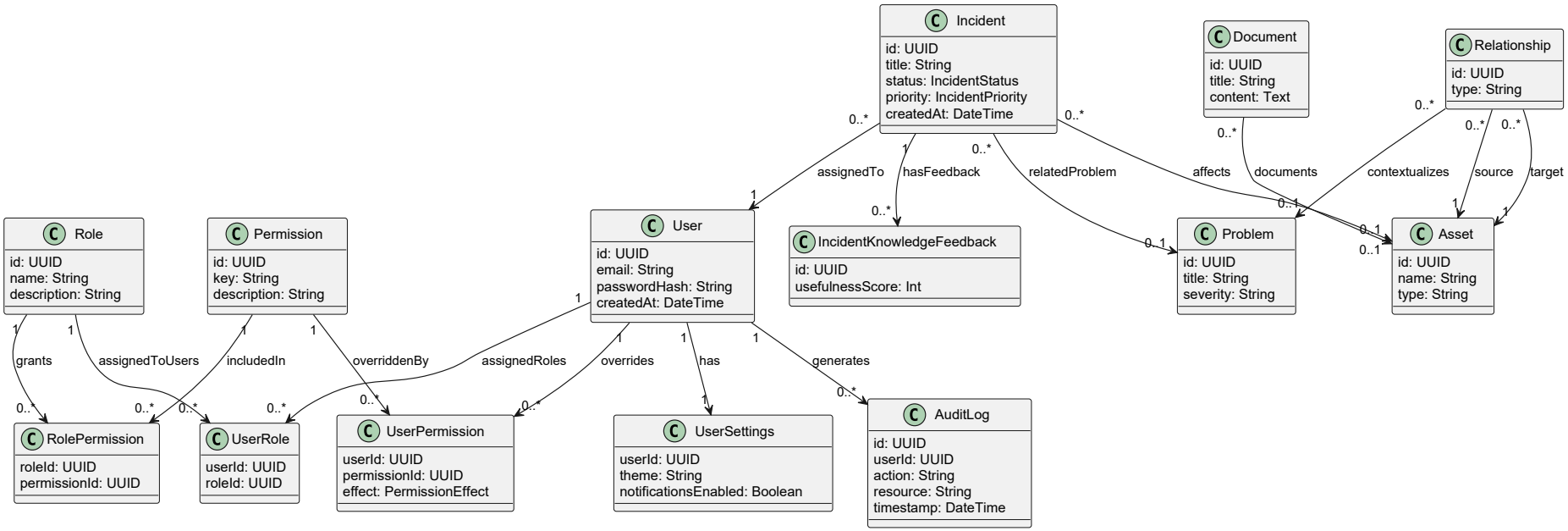

Figure 5.5: Class diagram of the MapIT domain model

5.6 Data Model and Relational Schema

Once the conceptual model is established, it must be translated into a relational schema. This part of the chapter should document the tables, primary keys, foreign keys, cardinalities, and integrity constraints used to preserve consistency in the database. If a database diagram is provided, it should confirm that the schema reflects the behavior described in the use case and class models.

5.7 Optional Complementary Diagrams

Additional diagrams such as activity, component, or deployment diagrams may be added if they bring real clarity to the project. For example, an activity diagram can explain a complex workflow, while a deployment diagram can clarify how the frontend, backend, and database are hosted. These diagrams should be included selectively so that they reinforce the design instead of cluttering the report.

To clarify the permission-governed workflow used in sensitive operations, Figure 5.6 shows the RBAC activity flow.

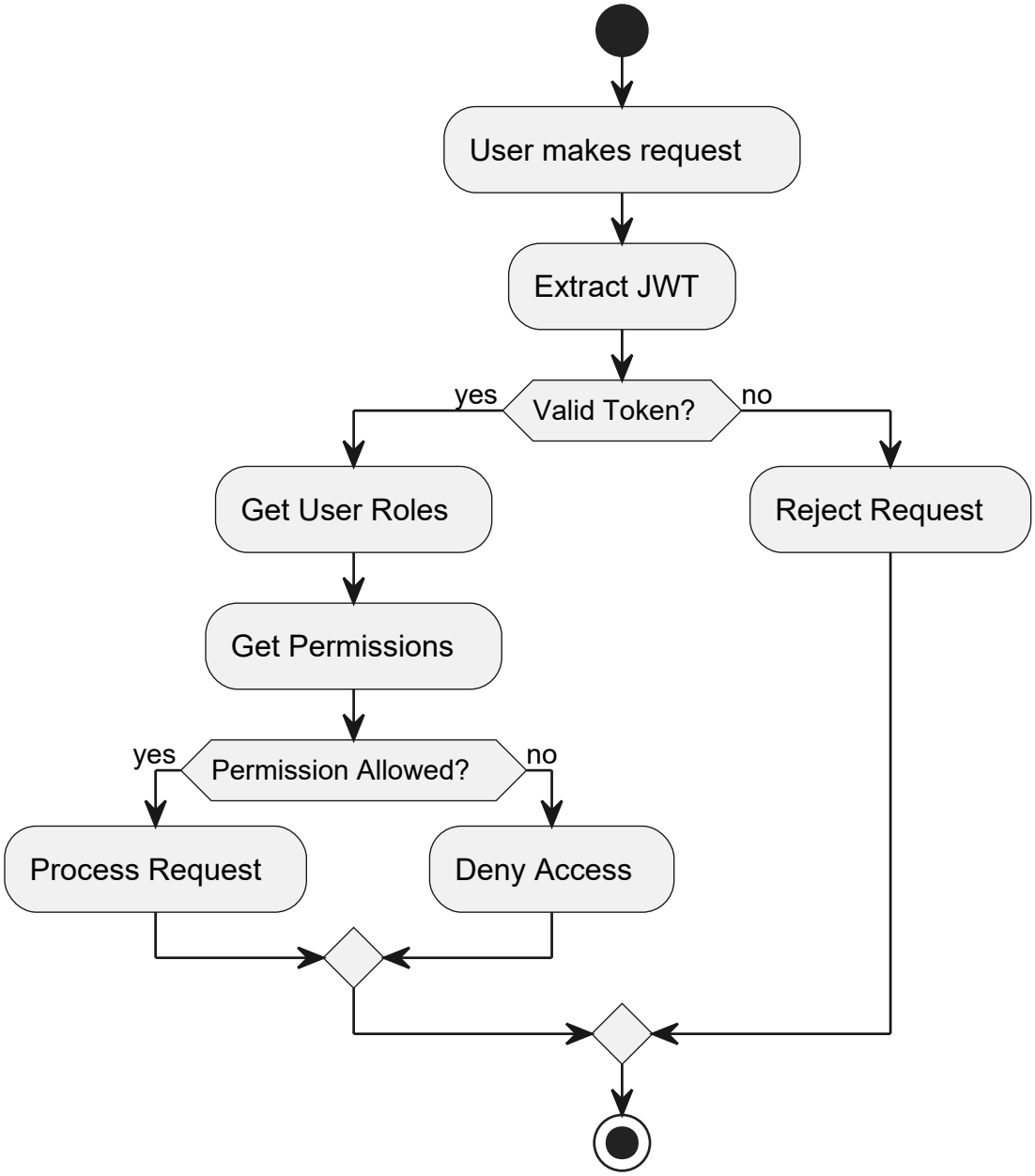

Figure 5.6: Activity diagram: RBAC decision flow

5.8 Chapter Conclusion

The modeling phase validates the structure of the proposed solution and makes the implementation task more explicit. By separating dynamic behavior, static structure, and data organization, the chapter demonstrates that MapIT has been designed in a disciplined way. The next chapter explains how this design was realized in the actual software stack.

Chapter 6

Implementation

6.1 Introduction

This chapter presents the realization of MapIT in the source code. It is intentionally project-specific: the implementation is not described as a generic web application, but as a Next.js 15 and Express.js platform that serves IT infrastructure documentation, incident tracking, asset inventory, relationship mapping, search, reporting, and access control.

6.2 Technical Stack and System Architecture

6.2.1 Monorepo structure

MapIT is organized as a workspace-based monorepo. The main workspace folders are:

- `apps/api`
- `apps/web`
- `apps/mobile`
- `packages/shared`

The most important root scripts are:

- `dev:api`, `dev:web`, `dev:mobile`
- `typecheck`, `test:api`, `smoke:frontend`, `ci`

This structure is important because it separates the operational backend, the browser interface, and the mobile prototype while keeping the project coherent.

6.2.2 Web implementation stack

The web interface uses Next.js 15 with the App Router, React 19, and TypeScript in strict mode. The styling relies on Tailwind CSS 3 and a utility-driven component system, while the state management layer uses Zustand. API calls are made through an Axios client with a 10-second timeout, `withCredentials` enabled, and a response interceptor that redirects to `/login` on HTTP 401. The project also uses Lucide icons, React Hook Form, Zod, SWR, TanStack React Query, and Recharts where the UI needs them.

6.2.3 Backend implementation stack

The backend is built on Express 5, Prisma ORM, PostgreSQL, JSON Web Tokens, and ldapjs for Active Directory authentication in production. The server structure is modular: each domain has its own route file under `apps/api/src/modules/`. There are dedicated modules for authentication, incidents, documents, dashboard summaries, assets, problems, relationships, search, reports, notifications, settings, and administration.

6.2.4 Data and persistence layer

The Prisma schema defines the real MapIT data model. The main entities include:

- `User`, `Incident`, `Document`, `Asset`, `Problem`, `Relationship`
- `Role`, `Permission`, `UserRole`, `RolePermission`, `UserPermission`
- `UserSettings`, `AuditLog`, and `IncidentKnowledgeFeedback`

This is the persistence layer that makes the platform suitable for technical documentation rather than only for temporary user input.

6.3 Authentication and Access Control

6.3.1 Login flow

The login route accepts a username and password, authenticates against Active Directory through ldapjs when the application is not in development mode, and supports test credentials when `DEV_MODE=true`. After successful authentication, the user is upserted in the database, assigned a default role, and issued a JWT valid for one day. The web login page posts to `/api/auth/login` and then redirects the user to the dashboard.

The login interface is shown in Figure 6.1.

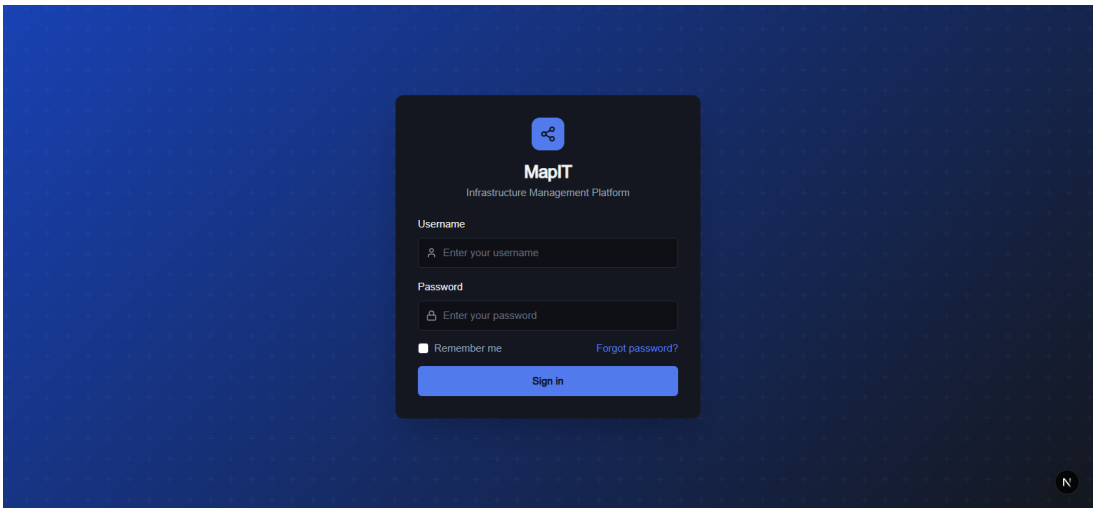

Figure 6.1: Login page

6.3.2 Session and proxy handling

The frontend does not talk directly to the backend hostname. It sends requests to the Next.js proxy path `/api/backend/[...path]`, which forwards the browser traffic to the API while keeping the public web application same-origin. The auth store also checks `/api/auth/session` to determine whether the session is still valid. This is a practical implementation pattern for a secure and deployment-friendly web application.

6.3.3 Role-based and permission-based access control

MapIT goes beyond a simple admin/viewer split. The backend bootstraps permissions and role templates for Admin, Manager, Operator, and Viewer, and it supports user-specific permission overrides. Middleware functions such as `requireAuth`, `requireRole`, and `requirePermission` protect the API. The permission catalog includes fine-grained actions, for example:

- `asset:create`, `incident:status:update`, `incident:assign`
- `relationship:create`, `report:view`, `settings:update-any`
- `user:manage`, `permission:manage`

Figure 6.2 illustrates the access-control administration interface used to manage these permission assignments.

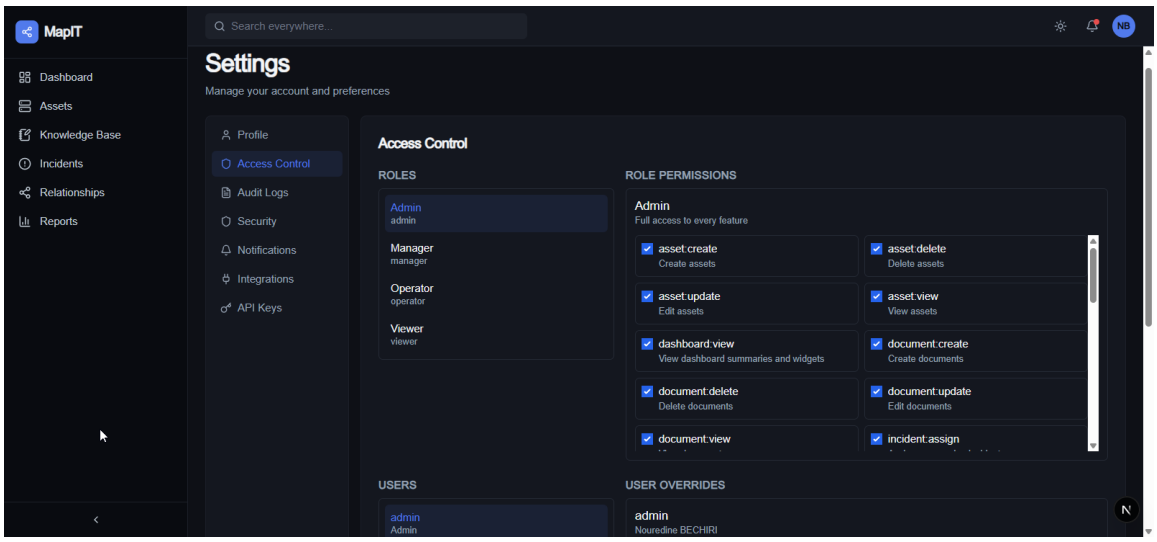

Figure 6.2: Access control management interface

6.4 Backend Modules and Business Functions

6.4.1 Incidents

The incident module is one of the core components of MapIT. It manages incident creation, status transitions, assignment, and knowledge suggestions. The model uses the statuses

`OPEN`, `IN_PROGRESS`, `BLOCKED`, `RESOLVED`, and `CLOSED`, together with priorities from `LOW` to `CRITICAL`. The detail screen in the web app allows the user to edit the title, description, assigned team, assignee, and status, and it also integrates related knowledge suggestions.

6.4.2 Assets and relationships

The assets module stores infrastructure components such as servers, virtual machines, network devices, storage nodes, and services. The relationship module creates directed connections between assets and returns the graph used by the relationship map page. In the frontend, the map supports dragging nodes, filtering by type, panning, and zooming. This is how MapIT makes infrastructure interdependence visible.

6.4.3 Problems and knowledge reuse

The problems module stores documented problems, affected assets, severity, status, solution text, and resolution dates. This module is the reusable knowledge layer of the application. It is exposed in the Problems page, where users can create, edit, filter, and delete problem entries according to their permissions. The global search module uses these records together with assets, incidents, and documents to produce a unified search result.

6.4.4 Search, reports, settings, and audit logs

The search page performs global search across all knowledge domains. The reports page summarizes assets by type and problems by severity and provides CSV/PDF export. The settings page manages profile data, notifications, theme preferences, access-control administration, and audit-log inspection. The audit trail stores actor, action, resource, and sanitized request metadata so that sensitive operations remain traceable.

The audit trail visualization in Figure 6.3 shows how operational actions remain traceable over time.

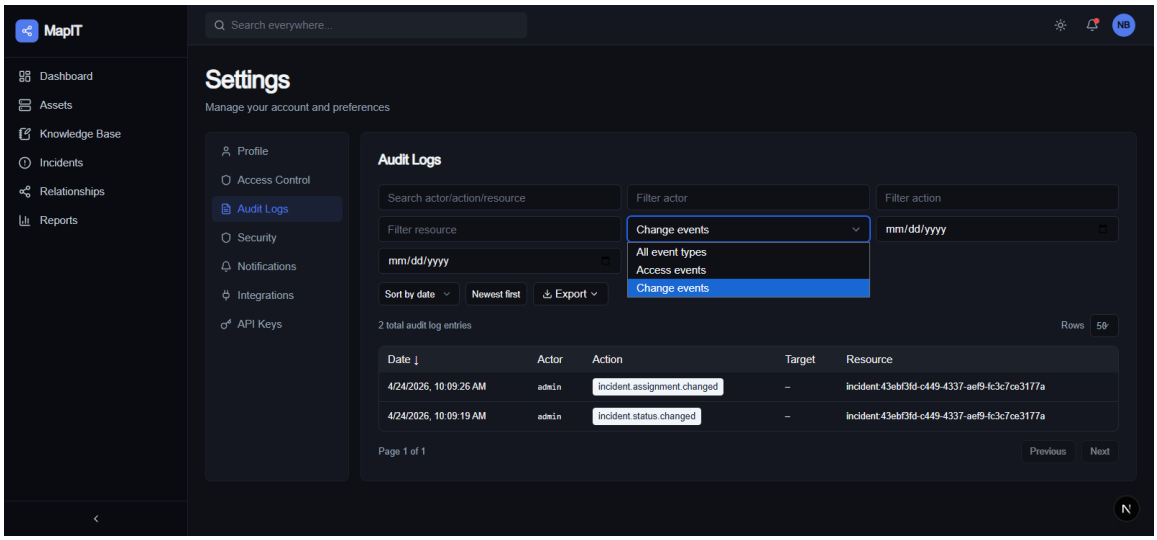

Figure 6.3: Audit logs interface

6.5 Realized User Interfaces

6.5.1 Login and shared layout

The login page contains a username field, a password field, inline error feedback, and a sign-in button. After authentication, the application redirects to the dashboard. The authenticated pages share a sidebar layout, which is used consistently on the dashboard, incidents, assets, problems, relationships, reports, and settings sections.

6.5.2 Dashboard

The dashboard is not a static welcome page. It loads summary data, assets, and problems in parallel and exposes export actions for CSV and PDF generation. The user can navigate from the dashboard to assets or problems directly, which makes it a control center for the infrastructure team.

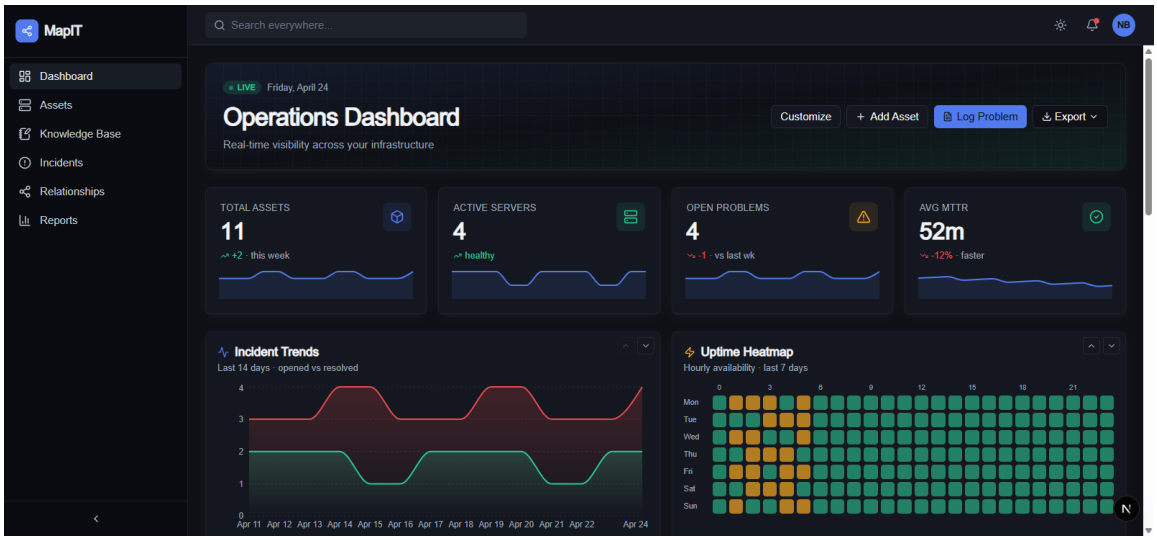

Figure 6.4: Dashboard interface

6.5.3 Incidents

The incidents list page supports search by title and description, creation of new incidents, and navigation to incident detail pages. The detail page supports editing, status updates, assignment changes, deletion, and knowledge suggestions. The page also shows status badges and priority badges, which are important for rapid operational reading.

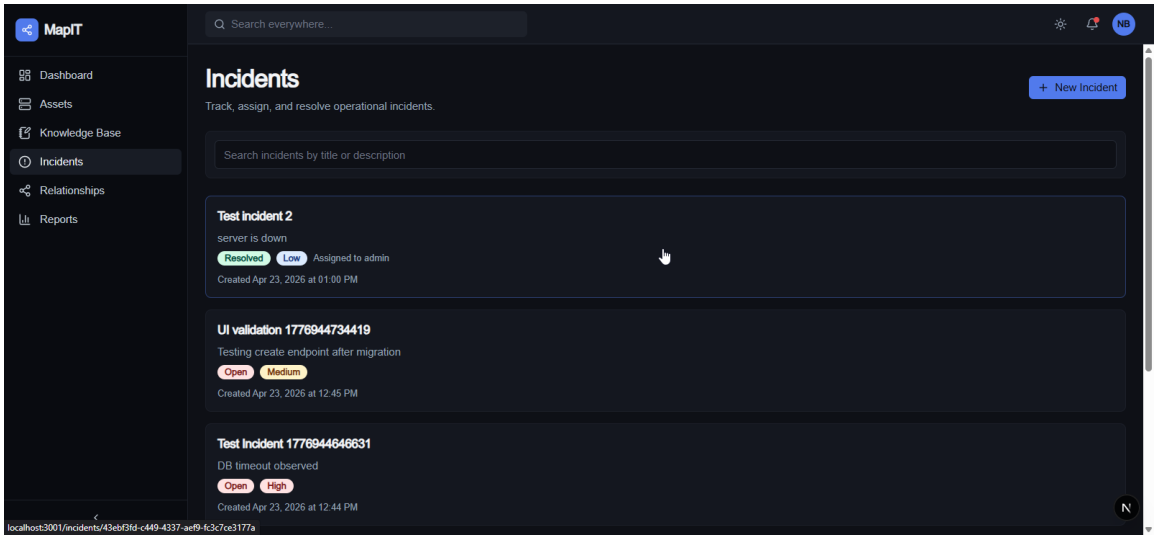

Figure 6.5: Incidents management interface

6.5.4 Assets

The asset inventory page supports filtering, pagination, bulk selection, and an asset creation form that can also create relationships immediately after the asset is saved. The asset detail page shows overview data, related problems, and related assets in separate tabs. The presence of a direct link to the relationship map reinforces the visual nature of the knowledge model.

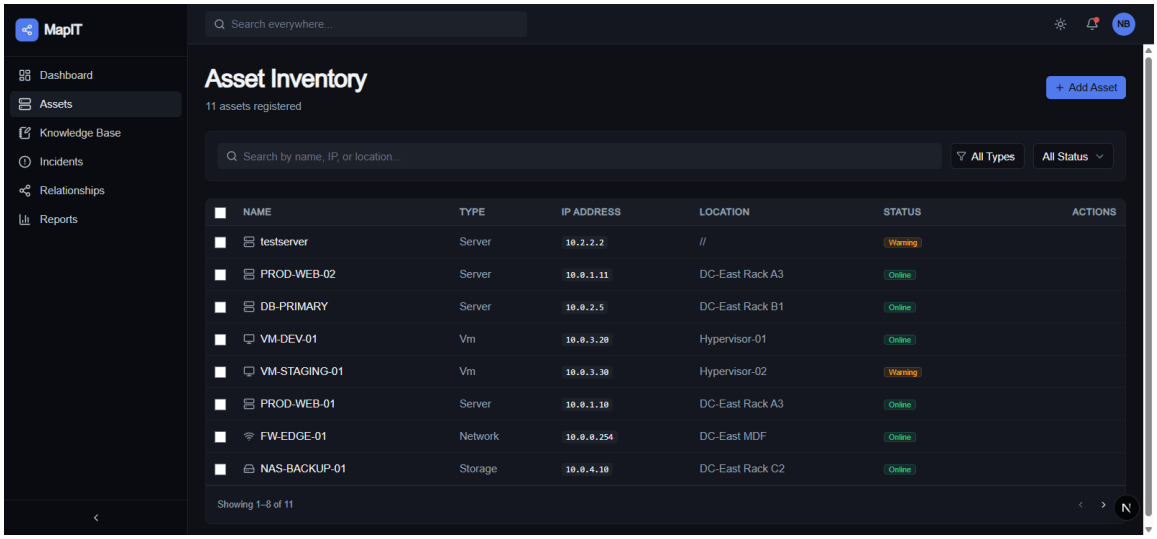

Figure 6.6: Assets inventory interface

6.5.5 Problems, relationships, search, reports, and settings

The problems page allows the user to manage problem records and associate them with affected assets. The relationships page visualizes the infrastructure graph and supports interactive layout editing. The search page searches across assets, incidents, problems, and documents in one place. The reports page summarizes operational data. The settings page exposes access control, audit logs, security, notifications, integrations, and API-key placeholders.

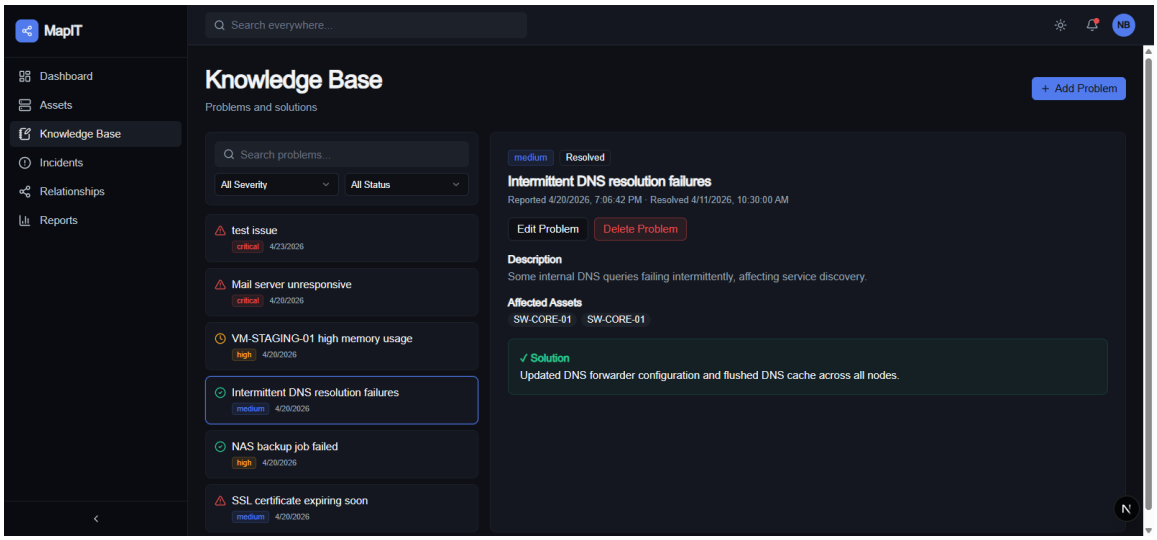

Figure 6.7: Problem and knowledge base interface

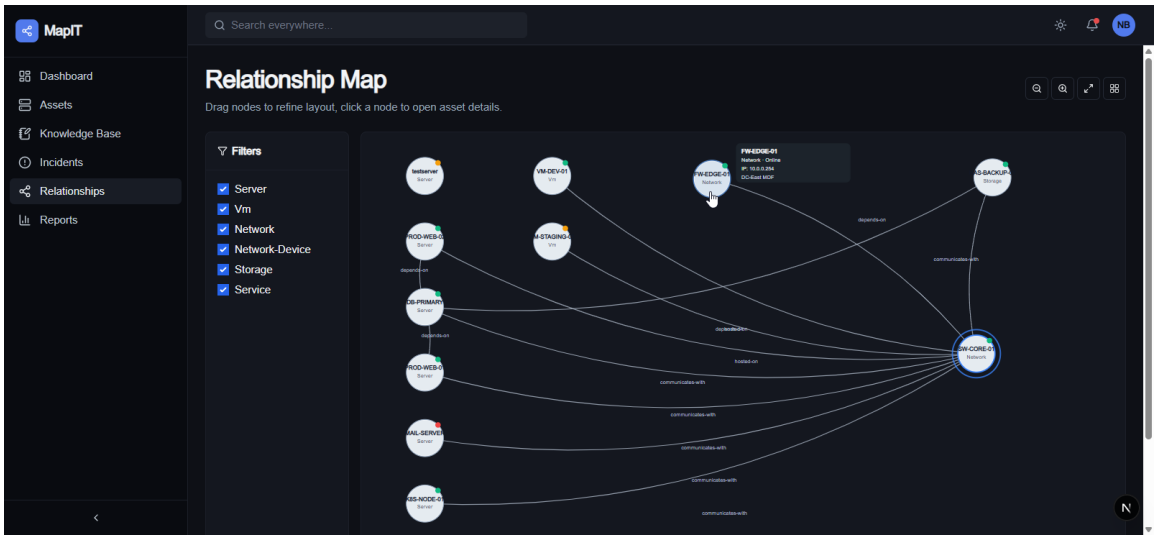

Figure 6.8: Relationship map interface

Figure 6.9: Global search interface

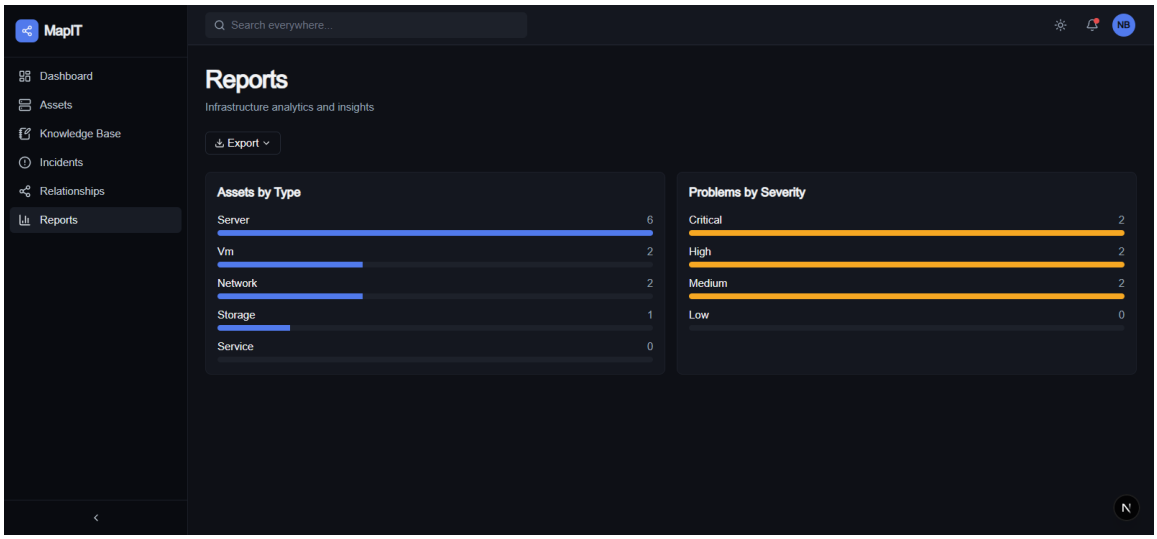

Figure 6.10: Reports interface

6.6 Deployment and Execution Notes

6.6.1 Local execution

The local development workflow is explicit: `npm run dev:api` starts the API on port 3000, `npm run dev:web` starts the web app on port 3001, and the root scripts support linting, type checking, integration testing, and smoke testing. This is consistent with the way the application is meant to be maintained during development.

6.6.2 Environment and configuration

The backend requires `DATABASE_URL`, `JWT_SECRET`, `AD_URL`, `AD_DOMAIN`, `CORS_ORIGIN`, `ADMIN_USERNAMES`, and `DEV_MODE`. The web layer uses the proxy base `/api/backend` and the session route `/api/auth/session`. These configuration points are evidence that the project is already organized for a real deployment scenario.

6.6.3 Production orientation

The README describes deployment readiness on Vercel for the web application, Docker for the API, and Expo builds for the mobile prototype. The implementation chapter should therefore present MapIT as a deployable system rather than as a code-only exercise.

6.7 Validation and Testing

6.7.1 Backend checks

The backend uses integration tests under the Node test runner, and Prisma generation is part of the backend lifecycle. The audit logger, permission middleware, and authentication route can all be exercised through API requests, which is a strong basis for validation.

6.7.2 Frontend checks

The web app supports TypeScript type checking, linting, Playwright end-to-end testing, and smoke-style validation. These checks are useful because MapIT's value depends not only on the backend data model but also on whether users can navigate the real pages effectively.

6.8 Chapter Conclusion

The implementation of MapIT is specific, structured, and aligned with the project's academic objective. It combines a secure backend, a modern web frontend, a normalized

data model, graph-based relationship mapping, global search, reporting, and access control. The next and final chapter can now synthesize the contribution of the project and outline future improvements.

General Conclusion

MapIT was designed to address a practical need that appears frequently in IT environments: the difficulty of keeping infrastructure documentation and incident knowledge in one reliable place. By combining a structured data model, role-aware access, and a clear web interface, the project demonstrates how a focused application can improve knowledge reuse and support operational continuity. The work also shows how theoretical concepts such as incident management, knowledge governance, and modeling translate into a coherent software solution.

From a practical point of view, the application establishes a foundation that can be extended with richer search, asset relationships, analytics, workflow automation, and stronger reporting features. These improvements would strengthen the product beyond the academic prototype and move it closer to a complete operational knowledge platform.

Bibliography

[1] Axelos. *ITIL Foundation: ITIL 4 Edition*. TSO, 2019.

[2] ISO/IEC. Iso/iec 20000-1: Information technology – service management, 2018. International standard.

[3] Ikujiro Nonaka and Hirotaka Takeuchi. *The Knowledge-Creating Company*. Oxford University Press, 1995.

[4] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. McGraw-Hill, 2019.

[5] SNTF. Société nationale des transports ferroviaires - site officiel, 2026. Consulted for the organization presentation.

[6] Wikipédia. Société nationale des transports ferroviaires, 2026. Consulted for historical and factual context.